NEXTTRADE

Enterprise Trading Platform

Technical Architecture & Design Documentation

Production-Ready Microservices Architecture

Version 1.0 — Pre-Production Release

Architecture: Microservices | Language: C# .NET 8

Stack: ASP.NET Core · RabbitMQ · Redis · PostgreSQL · HashiCorp Vault · Docker

1. Executive Summary

> *NextTrade is a production-grade, event-driven trading platform built on a distributed microservices architecture. It handles end-to-end trade lifecycle management — from secure authentication and order ingestion to real-time processing and execution — while enforcing enterprise-grade security, reliability, and scalability.*

The platform was designed to handle high-throughput financial order flows with strict guarantees around message delivery, security, and auditability — requirements common in regulated financial environments such as equity trading, fund management, and brokerage operations.

1.1 Business Problem Solved

- Financial trading platforms require zero message loss — an order cannot disappear between services
- Security is paramount: JWT signing keys must never be exposed, even to the services using them
- Login endpoints are prime targets for brute-force and credential stuffing attacks
- In distributed systems, a service going down must not lose committed data
- Full request traceability across services is required for compliance and debugging

1.2 Platform Highlights

Capability	Implementation	Industry Pattern
Zero-loss messaging	Transactional Outbox Pattern	Saga / Outbox
Secret management	HashiCorp Vault Transit Engine	Secret Zero / HSM
Brute-force protection	Dual-key Redis rate limiting	Token bucket / Sliding window
Async order processing	RabbitMQ event-driven pipeline	Event-Driven Architecture
Distributed tracing	Correlation ID propagated via headers + MQ	OpenTelemetry pattern
Clean architecture	CQRS + MediatR pipelines + SOLID	Clean/Onion Architecture

2. System Architecture Overview

NextTrade follows a microservices architecture where each service has a single, well-defined responsibility. Services communicate asynchronously via RabbitMQ, ensuring loose coupling and high resilience.

2.1 Microservices Breakdown

Service	Port	Responsibility	Key Tech
Authentication Service	5006	Login, JWT token generation, rate limiting	Vault Transit, Redis, RSA-2048
Order Service	5001	Order ingestion, validation, outbox publishing	CQRS, MediatR, PostgreSQL, RabbitMQ
Processing Service	5002	Order consumption, business validation, execution	RabbitMQ consumer, Strategy pattern

2.2 Infrastructure Components

Component	Technology	Role
Message Broker	RabbitMQ 3 (management UI on :15672)	Async event transport between Order ↔ Processing
Cache & Rate Store	Redis 7 (Alpine)	JWT token cache + rate limit counters + IP whitelist
Secret Management	HashiCorp Vault 1.15	JWT RSA-2048 key storage; private key never leaves Vault
Database	PostgreSQL 15	Orders table + OutboxMessages table (transactional)
Observability	Serilog + Seq (port :5341)	Structured logs with CorrelationId across services
DB Admin	pgAdmin 4 (port :5050)	PostgreSQL management UI
Containerization	Docker Compose	All services in a shared `trading-network` bridge

2.3 Network Topology

All containers run in a Docker bridge network named trading-network. Services communicate by container name DNS (e.g., rabbitmq:5672, redis:6379, vault:8200). Health checks with retry logic ensure dependent services start only after their dependencies are healthy.

```
Startup order (via depends_on + health checks):
```

```
Vault → Redis → RabbitMQ → Auth Service
                         → Order Service
          RabbitMQ → Processing Service
```

3. Authentication Service — Deep Dive

> *The Auth Service is the security gateway of the platform. It issues JWT tokens signed by HashiCorp Vault using RSA-2048 keys, with the private key never leaving Vault at any point.*

3.1 JWT Signing Flow (Vault Transit Engine)

Traditional JWT implementations store the private signing key in environment variables or config files — a significant security risk. NextTrade uses Vault's Transit Engine (a cloud HSM equivalent) where:

- The RSA-2048 private key is generated inside Vault and never exported
- Auth Service authenticates to Vault using AppRole (RoleId + SecretId)
- Auth Service sends the JWT payload to Vault's /v1/transit/sign endpoint
- Vault signs internally and returns only the signature — the private key never leaves
- Keys auto-rotate every 30 days (auto_rotate_period=720h); old versions retained until tokens expire

Vault Privilege Separation (Least Privilege):

Service	Vault Policy	Capabilities
Auth Service	auth-service-policy	transit/sign (create, update) + transit/keys (read)
Order Service	order-service-policy	transit/keys (read only) — cannot sign, only verify

3.2 Redis JWT Token Cache

To avoid Vault round-trips on every login, tokens are cached in Redis with a TTL matching the JWT expiry (60 minutes). The caching layer implements fail-open design:

- Cache HIT with sufficient TTL (> CacheRefreshThresholdMinutes): return cached token
- Cache HIT but TTL near expiry: treat as cache miss, generate fresh token
- Redis unavailable: generate fresh token from Vault without impacting availability
- Permissions change / session revoke: InvalidateAsync() deletes the Redis key

```
Redis key pattern: token-cache:{username}:{roles_hash}
```

3.3 Dual-Key Rate Limiting (Brute Force Defense)

The rate limiter protects the /api/auth/login endpoint using two independent counters in Redis:

Dimension	Redis Key	Limit	Window	Blocks
Per-IP	auth-rate-limit:ip:{ip}	20 requests	300 seconds	Password spraying from one machine
Per-Username	auth-rate-limit:user:{username}	5 requests	300 seconds	Botnet brute-force across IPs

Both limits must pass. Rejection returns HTTP 429 with Retry-After header. The middleware reads body twice safely using Request.EnableBuffering(). Username is normalized to lowercase to prevent case-based bypass (admin ≠ Admin ≠ ADMIN).

3.4 IP Whitelist

Trusted IPs (office networks, VPN ranges, trusted partners) are stored in a Redis SET and bypass rate limiting entirely. Whitelist is managed at runtime without service redeploy:

```
Redis key: ip-whitelist (SET data structure)
SMEMBERS ip-whitelist   → list all whitelisted IPs
SADD ip-whitelist {ip}  → add IP at runtime
```

Real client IP extraction handles X-Forwarded-For from load balancers and reverse proxies, using a known-proxies list to prevent IP spoofing via forged headers.

4. Order Service — Deep Dive

> *The Order Service is the entry point for all trade orders. It implements Clean Architecture with CQRS, MediatR pipeline behaviors, and the Transactional Outbox Pattern to guarantee zero message loss.*

4.1 Request Lifecycle (8 Steps)

1. Client sends POST /api/order/create with Bearer JWT token
2. UseAuthentication() middleware extracts kid from JWT header → calls JwtKeyResolver → fetches RSA public key from Vault → verifies RS256 signature
3. CorrelationIdMiddleware extracts or generates X-Correlation-ID, pushes to Serilog LogContext
4. OrderController maps OrderRequest → OrderCreatedCommand via AutoMapper
5. MediatR Pipeline: LoggingBehavior → ValidationBehavior (FluentValidation) → AuthorizationBehavior → OrderCreatedCommandHandler
6. CommandHandler creates Order entity + OutboxMessage, saves both atomically in a single PostgreSQL transaction
7. OutboxBackgroundService polls for pending outbox messages → publishes OrderPlacedEvent to RabbitMQ → marks message as processed
8. Returns HTTP 202 Accepted with OrderId

4.2 Transactional Outbox Pattern

This is the most critical reliability pattern in the system. Without it, the following failure scenario exists:

```
PROBLEM: Save order to DB → [service crashes] → Message never sent to RabbitMQ → Order lost forever
```

The Outbox Pattern solves this by making the message an integral part of the database transaction:

```
SOLUTION: BEGIN TRANSACTION
  INSERT INTO Orders (...)
  INSERT INTO OutboxMessages (payload=OrderPlacedEvent, status=PENDING)
COMMIT TRANSACTION ← atomic, all-or-nothing


OutboxBackgroundService (separate thread):
  SELECT * FROM OutboxMessages WHERE status = PENDING
  → Publish to RabbitMQ
  → UPDATE status = PROCESSED
```

Even if the service crashes after the transaction but before publishing, the OutboxBackgroundService will pick up the pending message on next poll. This guarantees at-least-once delivery.

4.3 CQRS + MediatR Pipeline

Pipeline Behavior	Responsibility	Pattern
LoggingBehavior	Log request/response, timing, errors	Decorator / Cross-cutting concern
ValidationBehavior	Run FluentValidation validators, throw on failure	Chain of Responsibility
AuthorizationBehavior	Check ICurrentUserService.IsAuthenticated	Decorator
OrderCreatedCommandHandler	Core business logic: create order + outbox	Command Handler

4.4 JWT Verification (Order Service Side)

Order Service uses the same Vault-signed public keys to verify tokens WITHOUT needing Vault signing capability:

- JwtKeyResolver reads kid from JWT header (key ID)
- Fetches public key from Vault /v1/transit/keys/jwt-signing-key (read-only policy)
- Imports PEM using ImportSubjectPublicKeyInfo → creates RsaSecurityKey
- ASP.NET middleware verifies RS256 signature + exp + iss + aud claims
- Key rotation transparent: kid in JWT header identifies which version to use

5. Processing Service — Deep Dive

> *The Processing Service is a pure consumer — it has no HTTP endpoints. It runs as a background service listening to RabbitMQ, and implements the full order validation and execution pipeline using SOLID principles and the Strategy Pattern.*

5.1 Message Consumption Flow

9. RabbitConsumerService (BackgroundService) starts and declares queue order-placed-events
10. Sets QoS prefetch = 1 (process one message at a time, prevents overload)
11. Message arrives → extract CorrelationId from headers → push to Serilog context
12. Create new DI scope per message (prevents memory leaks with scoped services)
13. OrderPlacedEventConsumer.ConsumeAsync() is called
14. Step A: IOrderValidator.Validate() → if invalid, log warning and discard (no retry for business failures)
15. Step B: IOrderExecutor.ExecuteAsync() → simulate market execution
16. BasicAck() → message removed from queue on success
17. BasicNack(requeue: true) → message requeued on infrastructure failure

5.2 SOLID Architecture in Processing Service

Interface	Purpose	SOLID Principle
IOrderValidator	Validate order business rules	DIP: depend on abstraction, not concrete class
IOrderExecutor	Execute trade in market	OCP: swap execution strategies without changing consumer
IMarketDataProvider	Check market health, get prices	SRP: only provides market data
IClientPositionProvider	Validate client has position for SELL	ISP: focused, minimal interface
IOrderExecutionEventPublisher	Publish execution results downstream	DIP: injectable publisher

5.3 Error Classification Strategy

Exception Type	Meaning	Action
OrderValidationException	Business rule violated (invalid order)	Discard — do not retry, order is inherently invalid
OrderExecutionException	Execution failed (market closed, no position)	Handle based on ErrorCode: some retry, some discard
Generic Exception	Infrastructure failure (DB down, network)	Requeue — retry when infrastructure recovers

5.4 Execution Event Publishing

After successful execution, Processing Service publishes downstream events to two queues:

- order-executed-events: Consumed by Notification, Reporting, Settlement, Compliance services
- order-failed-events: Consumed by Alerting and Retry services

Each published message carries the CorrelationId, enabling full end-to-end request tracing from HTTP login → order create → order process → execution.

6. Observability & Distributed Tracing

6.1 Correlation ID Flow

Every request generates a unique CorrelationId (UUID) that flows through the entire system:

```
HTTP Request → X-Correlation-ID header (generated if absent)
  → Serilog LogContext.PushProperty("CorrelationId", id)
  → RabbitMQ message properties headers["X-Correlation-ID"]
  → Processing Service extracts from message properties
  → All logs in all services tagged with same CorrelationId
  → HTTP Response includes X-Correlation-ID header
```

This enables a developer or support engineer to search Seq for a single CorrelationId and see the complete lifecycle of one order across all three services in chronological order.

6.2 Structured Logging with Serilog + Seq

- All services use Serilog with structured JSON output
- Seq UI (port 5341) provides searchable, filterable log explorer
- Log enrichers: FromLogContext adds CorrelationId, request metadata automatically
- Log levels: Debug in Development, Information in Production
- Seq query example: CorrelationId = "abc123" shows complete order journey

6.3 Health Checks

Service	Health Check Command	Retry Strategy
Redis	redis-cli ping	Interval: 10s, Timeout: 5s, Retries: 5
RabbitMQ	rabbitmq-diagnostics ping	Interval: 10s, Timeout: 5s, Retries: 5
Vault	vault status	Interval: 10s, Timeout: 5s, Retries: 10, Start: 30s

7. Design Patterns & Architecture Decisions

7.1 Patterns Used

Pattern	Where Used	Why Chosen
Transactional Outbox	Order Service DB + Background Service	Guarantees at-least-once delivery without 2-phase commit
CQRS	Order Service (Command/Query separation)	Scales reads and writes independently; clear intent
MediatR Pipeline	Order Service request pipeline	Cross-cutting concerns (logging, validation) without modifying handlers
Strategy Pattern	IOrderExecutor, IOrderValidator implementations	Swap execution/validation logic without touching consumers
Repository Pattern	IOrderRepository, IOutboxRepository	Abstracts DB access; enables testability
Factory Pattern	IOrderFactory creates Order + OutboxMessage	Centralizes object creation logic
Decorator Pattern	MediatR Pipeline Behaviors	Adds behavior (logging, auth) non-invasively
Observer Pattern	Event publishing after order execution	Decouples producer from multiple downstream consumers
Fail-Open Design	Redis token cache, Redis rate limiter	Redis failure → system keeps working (graceful degradation)

7.2 SOLID Principles Applied

Principle	Example in Codebase
S — Single Responsibility	RedisRateLimiter only rate-limits. RedisTokenCacheService only caches. OrderValidator only validates.
O — Open/Closed	IOrderExecutor: add new execution strategy (limit orders, TWAP) without modifying consumer
L — Liskov Substitution	MockMarketDataProvider can replace RealMarketDataProvider in tests without breaking behavior
I — Interface Segregation	IMarketDataProvider (market data only), IClientPositionProvider (positions only) — not one fat interface
D — Dependency Inversion	All services depend on interfaces (IOrderValidator, IOrderExecutor) injected via DI container

7.3 Clean Architecture Layers

- Domain Layer: Order entity, OutboxMessage, domain exceptions (OrderValidationException, OrderExecutionException), value objects
- Application Layer: CQRS commands/queries, MediatR handlers, interfaces (IOrderRepository, IOrderFactory), pipeline behaviors
- Infrastructure Layer: Repository implementations, RabbitMQ publisher/consumer, Vault client, Redis services
- API Layer: Controllers, middleware (CorrelationId, RateLimiting), AutoMapper profiles
- Shared Layer: Common events (OrderPlacedEvent), constants, CorrelationId context, extension methods

8. Production Readiness Checklist

8.1 Security

- ✅ JWT private key stored in Vault — never in config or environment variables
- ✅ AppRole authentication to Vault (no root token in production)
- ✅ Dual-key rate limiting prevents brute force and credential stuffing
- ✅ IP whitelist for trusted networks stored in Redis (no redeploy needed)
- ✅ RSA-2048 with auto-rotation every 30 days
- ✅ Least privilege: Order Service cannot sign JWTs, only verify
- ✅ Redis as shared state ensures rate limits work across multiple Auth Service instances

8.2 Reliability

- ✅ Transactional Outbox — no message loss even if service crashes post-commit
- ✅ Manual message acknowledgement (BasicAck/BasicNack) — no auto-loss on consumer crash
- ✅ RabbitMQ durable queues + persistent messages survive RabbitMQ restarts
- ✅ Health checks with retry logic prevent cascading startup failures
- ✅ Fail-open Redis design — token cache failure does not block logins
- ✅ QoS prefetch=1 prevents consumer overload during traffic spikes

8.3 Scalability

- ✅ Stateless services — Auth and Order Service can be horizontally scaled
- ✅ Redis shared state — rate limits consistent across multiple Auth Service pods
- ✅ RabbitMQ enables competing consumers — multiple Processing Service instances
- ✅ PostgreSQL connection pooling via Npgsql

8.4 Maintainability

- ✅ SOLID principles throughout — easy to extend without breaking changes
- ✅ MediatR pipeline behaviors — add cross-cutting concerns without touching handlers
- ✅ Interfaces everywhere — easy to mock in unit tests
- ✅ Structured logging with CorrelationId — debug any issue in minutes

- ✅ Docker Compose with startup scripts (start.sh / start.bat) — one-command deployment

9. Expected Interview Questions & Strong Answers

> ⚠ *Study these carefully. These are the questions an SDE3 interviewer WILL ask about this system.*

Q1: Why did you use HashiCorp Vault instead of storing the JWT key in environment variables?

> ★ *This is a security architecture question — answer with depth.*

Environment variables are readable by any process with the same OS user, visible in process lists, often leaked in error messages, and logged in CI/CD pipelines. Vault's Transit Engine acts as a software HSM — the private key is generated inside Vault and never exported. Signing happens inside Vault; we receive only the signature. Additionally, Vault provides key versioning (supports rotation), audit logging of every signing operation, fine-grained ACL (Auth Service can sign, Order Service can only read public keys), and lease-based access tokens that expire. This is the same model used by AWS KMS, Azure Key Vault, and GCP Cloud KMS in production financial systems.

Q2: Explain the Transactional Outbox Pattern and why it's necessary.

> ★ *Critical distributed systems question — this shows system design maturity.*

Without the outbox, we have a dual-write problem: save to DB and publish to RabbitMQ are two separate operations. If the process crashes between them, the DB has the record but the message was never sent — the order is silently lost. The outbox solves this by making the message part of the same database transaction. We write OrderPlacedEvent to an OutboxMessages table inside the same transaction as the Order record. A separate background service (OutboxBackgroundService) polls for pending outbox messages and publishes them to RabbitMQ, then marks them as processed. If publishing fails, the message stays pending and will be retried. This guarantees exactly-once write + at-least-once delivery without distributed transactions (2PC).

Q3: How does your rate limiter handle multiple instances of Auth Service?

> ★ *Scalability question — many candidates miss this.*

In-memory rate limiting breaks with horizontal scaling because each instance has its own counter — an attacker can bypass a 5-request limit by hitting 5 different instances. Our rate limiter uses Redis as shared state. The counter is a Redis key with an expiry (TTL). StringIncrement is atomic in Redis (single-threaded command execution), so there are no race conditions. All Auth Service instances share the same Redis counter, making the rate limit globally consistent regardless of how many pods are running.

Q4: What is dual-key rate limiting and why not just rate limit by IP?

> ★ *Security depth question.*

IP-only rate limiting fails against botnets where the attacker uses thousands of different IPs — each IP stays under the limit while one username gets hammered. Username-only rate limiting fails in office environments where many legitimate users share one NAT IP — one noisy neighbor could lock out the entire office. Dual-key rate limiting applies both: per-IP (20 requests per 5 minutes) catches password spraying from one machine, and per-username (5 requests per 5 minutes) catches distributed brute force. Both must pass. This is the same approach used by major financial institutions and is recommended by OWASP for authentication endpoints.

Q5: How does the Correlation ID flow across services in your system?

> ★ *Observability / distributed systems question.*

The HTTP request arrives with or without an X-Correlation-ID header. CorrelationIdMiddleware generates one if absent and stores it in a thread-local AsyncLocal<string> via CorrelationIdContext. Serilog LogContext.PushProperty("CorrelationId") attaches it to every log line for that request. When Order Service publishes to RabbitMQ, the publisher reads the CorrelationId from context and writes it to the message headers (IBasicProperties.Headers["X-Correlation-ID"]). Processing Service extracts it from message properties on arrival and pushes it to its own Serilog context. The result: search for any CorrelationId in Seq and see the complete timeline across all three services.

Q6: Why use RabbitMQ over direct HTTP calls between Order and Processing Service?

> ★ *Architecture tradeoff question.*

Direct HTTP creates temporal coupling — if Processing Service is down, Order Service fails too. RabbitMQ decouples them in time: Order Service publishes and returns immediately; Processing Service consumes when ready. This gives us: (1) Back-pressure handling — if processing is slow, messages queue up without dropping; (2) Independent scaling — run 10 Processing Service instances consuming from the same queue; (3) Durability — durable queue + persistent messages survive restarts; (4) Retry semantics — BasicNack with requeue=true retries on failure. The trade-off is eventual consistency: the client gets 202 Accepted (not 200 OK with execution result). This is acceptable in trading where order placement and execution are inherently separate events.

Q7: Walk me through what happens when Redis goes down.

> ★ *Resilience and failure mode question.*

Three Redis use cases, three failure behaviors: (1) Token Cache — RedisTokenCacheService wraps all Redis operations in try/catch and returns null on failure. The Auth Service detects cache miss and generates a fresh token from Vault. Login succeeds with slightly higher latency. (2) Rate Limiting — the middleware wraps the rate limit check in try/catch. On any Redis failure, it logs a warning and calls await _next(context) — fail-open. The login endpoint stays available; the risk of a brief rate-limit bypass is accepted over a full outage. (3) Whitelist Check — same fail-open: if Redis is down and we can't check the whitelist, we proceed without whitelist bypass (conservative). Fail-open is a conscious design choice for availability over strict rate-limiting during Redis failure.

Q8: How does JWT key rotation work without downtime?

> ★ *Advanced security operations question.*

Vault auto-rotates the RSA key every 30 days. When rotation happens, the old key version is NOT deleted — it is retired (min_decryption_version is not changed until all tokens using that version expire). Each JWT carries a kid (key ID) claim in the header that identifies which key version signed it. Order Service's JwtKeyResolver reads the kid value, fetches the corresponding public key from Vault's key versions response, and uses it for verification. New tokens are signed with the latest key version. Old tokens (signed with previous version) continue to verify with the old public key until they expire (60 minutes). There is zero downtime during rotation.

Q9: Why did you use MediatR pipeline behaviors instead of filters or middleware?

> ★ *Clean architecture design question.*

ASP.NET filters and middleware operate at the HTTP layer and have no awareness of the command/query type. MediatR pipeline behaviors are command-aware — they can inspect the specific command type and apply different logic. Our AuthorizationBehavior uses an ICommandAuthorizationRegistry to look up which roles are required for each specific command type. ValidationBehavior knows which FluentValidation validators apply to the current command. These behaviors are also reusable across commands without code duplication, testable in isolation without HTTP stack, and composable in a clear order. The pipeline pattern is essentially a chain of responsibility where each behavior decides whether to pass control to the next.

Q10: What would you change to make this truly production-ready at scale?

> ★ *This shows senior-level thinking — always have an improvement list.*

- Add an API Gateway (YARP or Ocelot) for routing, request aggregation, and central rate limiting

- Replace polling outbox with PostgreSQL LISTEN/NOTIFY for lower latency message dispatch
- Add Dead Letter Queue (DLQ) in RabbitMQ for messages that fail after N retries
- Implement distributed tracing with OpenTelemetry + Jaeger for trace visualization (not just log correlation)
- Add circuit breaker (Polly) around Vault calls — if Vault is slow, degrade gracefully
- Move from Docker Compose to Kubernetes for production orchestration (liveness/readiness probes, HPA)
- Add idempotency keys on Order creation to prevent duplicate orders from client retries
- Add PostgreSQL read replicas and read/write splitting for query scalability

10. Technology Stack Reference

Category	Technology	Version	Purpose
Runtime	C# .NET 8	8.0	All microservices
Web Framework	ASP.NET Core	8.0	HTTP APIs + Middleware pipeline
CQRS	MediatR	12.x	Command/Query routing + Pipeline behaviors
Validation	FluentValidation	11.x	Input validation with rich rule sets
Mapping	AutoMapper	12.x	DTO ↔ Domain object mapping
ORM	Dapper + Npgsql	Latest	Lightweight SQL with PostgreSQL
Message Broker	RabbitMQ	3-management	Async event transport
Cache	Redis	7-alpine	Token cache + Rate limiting state
Database	PostgreSQL	15	Orders + Outbox persistent store
Secret Manager	HashiCorp Vault	1.15.0	JWT key management (Transit engine)
Logging	Serilog + Seq	Latest	Structured logging + UI
Containerization	Docker + Compose	Latest	Full stack orchestration
JWT	System.IdentityModel.Tokens.Jwt	7.x	Token creation + validation

Quick Service URLs (Local Docker)

Service	URL
Auth Service API + Swagger	http://localhost:5006/swagger
Order Service API + Swagger	http://localhost:5001/swagger
Processing Service	http://localhost:5002 (no endpoints, consumer only)
RabbitMQ Management	http://localhost:15672 (guest/guest)
Seq Log Explorer	http://localhost:5341
pgAdmin	http://localhost:5050 (admin@example.com / admin)
Vault UI	http://localhost:8200